Entraînements HoopNet_2 et HoopNet_3

Détection de cerceaux par ellipse : ce qui a changé, comment le réseau apprend, ce que disent les résultats

Partie vision du projet de vol autonome d'un drone Tello
Code : ~/Documents/tello · journaux : logs/HoopNet_2, logs/HoopNet_3

1. Où se situent V2 et V3

Ce document fait suite au premier rapport (rapport_vision_cnn.pdf), qui décrivait le modèle à boîtes englobantes. Depuis, le réseau ne prédit plus une boîte mais une **ellipse**, et cinq entraînements se sont succédé. Chaque version ne change qu'un petit nombre de choses, pour pouvoir attribuer un effet à une cause.

Version	Ce qui change	Données (cerceaux / vides)
HoopNet_0	Boîtes englobantes, entrée 256×192	58 / 19 (couloir)
HoopNet_1	Ellipses, entrée 320×240, focale approchée (683 px)	408 / 207 (couloir + cage)
HoopNet_2	Revue de 28 images mal étiquetées, 1 corde de plus, **vraie calibration** (927 px)	408 / 207
HoopNet_3	**Règle du tiers** : cerceau visible à moins de 1/3 = image vide (34 images)	374 / 241
HoopNet_4	Découpage entraînement / validation stable (§7). Entraînement à relancer	374 / 241

Les deux versions étudiées ici partagent le même code d'apprentissage. Elles ne diffèrent que par les **étiquettes** des images : V2 corrige des erreurs d'annotation, V3 applique une règle de décision sur les cerceaux à peine visibles. C'est un bon exemple de ce qui pèse le plus sur un réseau entraîné avec peu de données : la qualité et la cohérence des étiquettes.

2. Pourquoi une ellipse plutôt qu'une boîte

Le cerceau est un **cercle**. Une caméra (modèle sténopé) transforme toujours un cercle en **ellipse** exacte : l'ellipse n'est pas une approximation, c'est l'image réelle du cercle, et sa déformation contient la position et l'orientation du cerceau. Une boîte, au contraire, mélange taille et angle de vue.

Paramètre de l'ellipse	Ce qu'on en déduit pour le vol
Centre	Correction de cap et d'altitude
Grand axe	Distance : $d \approx f \cdot D$ / grand axe (D = 68 cm, f = 927 px). Il ne rétrécit pas quand le cerceau tourne, contrairement à la largeur d'une boîte
Rapport B/A (petit axe / grand axe)	≈ cos de l'angle entre la ligne de visée et l'axe du cerceau
Inclinaison θ	Autour de quel axe le cerceau est tourné

2.1 Ce que le réseau doit prédire

Pour chaque image, le réseau produit 7 nombres : un score de présence, puis 6 paramètres d'ellipse. Toutes les longueurs sont divisées par la **largeur** W de l'image (et non x par la largeur et y par la hauteur, ce qui déformerait les ellipses) :

```
cible = [ cx / W,  cy / W,  2A / W,  B / A,  cos 2θ,  sin 2θ ]
          centre            grand axe  rapport   inclinaison
```

- **B/A plutôt que B** : le rapport est directement l'information utile (angle de vue) et reste entre 0 et 1, ce qui garantit que le petit axe ne dépasse jamais le grand.
- **(cos 2θ, sin 2θ) plutôt que θ** : une ellipse tournée de 180° est la même ellipse, et θ passe brutalement de 179° à 0° pour deux ellipses presque identiques. Le réseau apprendrait mal

cette discontinuité. Avec le double angle, deux ellipses voisines ont toujours des codages voisins.

- **Les cordes** (cerceau présent mais réduit à un arc, ellipse impossible à tracer) comptent comme présentes, sans cible d'ellipse : la perte d'ellipse est simplement masquée pour elles.

3. Les données de V2 et V3

3.1 HoopNet_2 : corriger les étiquettes

Après HoopNet_1, on a passé toutes les images dans le modèle et listé celles où il **contredisait** l'annotation (28 images, fichier captures/a_verifier.txt). La plupart des « faux positifs » étaient en fait des images rangées dans no_hoop/ alors qu'un cerceau y était visible : le réseau avait raison et l'étiquette tort. Après revue, 6 images vides sont devenues des cerceaux, 4 cerceaux sont devenus des vides, et une ellipse aberrante (2 749 px de grand axe pour une image de 960 px) a été transformée en corde.

> **Méthode.** utiliser un premier modèle pour retrouver les erreurs d'annotation est une technique classique. Le modèle n'est pas un juge infaillible, mais ses désaccords avec l'annotation pointent vers les images à revoir.

V2 est aussi le premier entraînement avec la **vraie calibration** de la caméra (39 images de damier, erreur de reprojection 0,23 px). La focale mesurée, 927 px, est bien plus grande que les 683 px déduits des 82,6° annoncés par DJI : le flux vidéo 960×720 n'utilise que le centre du capteur (champ diagonal réel : 65,9°). Avec la valeur approchée, une rotation simulée de 10° correspondait en réalité à environ 7°.

3.2 HoopNet_3 : la règle du tiers

La revue a fait apparaître une incohérence : des arcs de cerceau presque identiques étaient étiquetés tantôt « cerceau », tantôt « vide ». Un réseau ne peut pas avoir raison sur les deux. La règle retenue : **si moins d'un tiers du contour du cerceau est dans l'image, l'image est vide**. Elle évite un modèle qui se déclenche sur le moindre bout d'arc. Le tiers est mesuré automatiquement en échantillonnant 64 points sur l'ellipse annotée.

34 images sont passées en vide (30 ellipses sous le tiers et les 4 cordes), et le seuil de l'augmentation (min_visible) est passé de 0,30 à 0,34 : un zoom ou une rotation ne fabrique jamais un « cerceau » moins visible que ce que la règle accepte. Conséquence pour le vol : tout près du cerceau, juste avant la traversée, le modèle dira « vide » ; la traversée se fait donc à l'aveugle une fois le drone engagé.

	HoopNet_2	HoopNet_3
Cerceaux / vides (total)	408 / 207, dont 4 cordes	374 / 241, aucune corde
Entraînement	492 images (326 cerceaux, 166 vides)	488 images (299 cerceaux, 189 vides)
Validation	123 images (82 cerceaux, 41 vides)	127 images (75 cerceaux, 52 vides)
dont ellipses aplaties (B/A < 0,85)	26	17

4. Les augmentations géométriques exactes

Chaque image d'entraînement est modifiée au hasard à chaque lecture. Pour les transformations qui déplacent le cerceau, l'ellipse annotée doit être transformée **exactement** de la même façon, sinon on apprend au réseau des cibles fausses.

Transformation	Comment l'ellipse suit	Ce qu'elle simule
Miroir, zoom, décalage	Forme paramétrique $x = c + M\,u$: on applique la transformation affine à c et M, puis une décomposition SVD redonne A, B, θ	Cerceau à gauche / à droite, plus ou moins loin
Rotation de caméra $H = K\,R\,K^{-1}$	Forme conique : $C' = H^{-T}\,C\,H^{-1}$ (une conique reste une conique)	Le drone tourne (lacet ±12°, tangage ±8°, roulis ±12°)
Collage sur un autre fond	Inchangée (seul le décor change)	Empêche d'apprendre le décor
Couleur, flou, bruit, JPEG, masque	Inchangée	Contre-jour, bougé, flux Wi-Fi

Une rotation **pure** de la caméra ne crée pas de parallaxe : tout le décor bouge comme le cerceau, et l'homographie $K\,R\,K^{-1}$ est exacte pour l'image entière. Un **déplacement** du drone, lui, ne peut pas être simulé ainsi, car le cerceau et le mur du fond, à des profondeurs différentes, bougeraient différemment : seules de vraies captures prises de nombreuses positions couvrent ce cas.

Ces transformations ont été vérifiées sur 300 ellipses aléatoires : les points du contour transformé tombent sur l'ellipse calculée avec un écart maximal de 10^{-12} pixel, c'est-à-dire la précision de la machine.

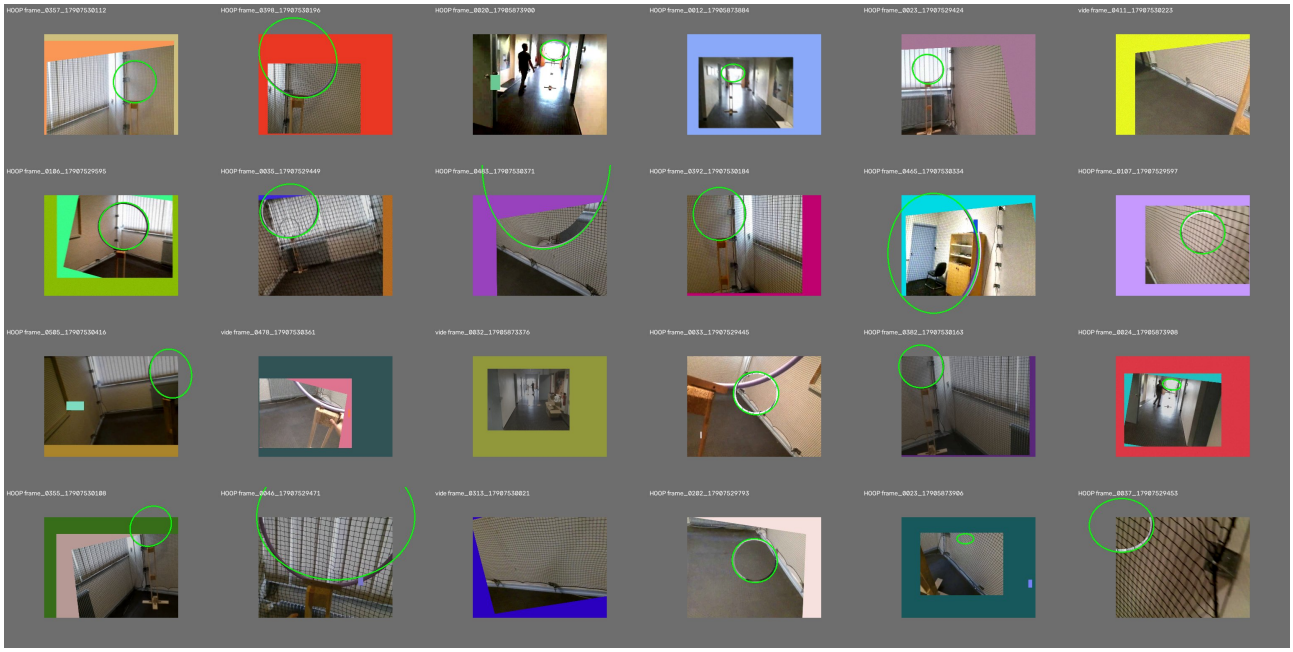


Figure 1 – Images vues par le réseau à l'entraînement. Les quadrilatères inclinés viennent des rotations simulées ; l'ellipse verte suit le cerceau dans chaque cas.

5. Le réseau, la perte et l'entraînement

5.1 Architecture

Étape	Opération	Forme du tenseur	Paramètres
Entrée	lot de 16 images 320×240	(16, 3, 240, 320)	—
Backbone	MobileNetV3-small pré-entraîné (pas total 32)	(16, 576, 8, 10)	0,93 M
Réduction	Conv 1×1 576→64 + BatchNorm + ReLU	(16, 64, 8, 10)	37 k
Dense	Aplatir (5 120) → Linear 256 + ReLU	(16, 256)	1,31 M
Sortie	Linear 256 → 7, puis activations	(16, 7)	1,8 k

On garde la grille spatiale 8×10 jusqu'à la couche dense au lieu de la moyenner : la position du cerceau dans l'image en dépend. Les activations de sortie bornent chaque grandeur : centre dans [−0,5 ; 1,5] (il peut sortir de l'image), grand axe dans]0 ; 2,5], rapport B/A dans]0 ; 1[par une sigmoïde, cos 2θ et sin 2θ dans [−1 ; 1] par une tangente hyperbolique. Au total, 2,28 millions de paramètres.

5.2 La perte

```
perte = BCE(présence)                                          (toutes les images)
      + 5 × [ SmoothL1(centre)                                 (seulement si une ellipse est annotée)
            + SmoothL1(log grand axe)          <- erreur RELATIVE, donc relative sur la distance
            + SmoothL1(B/A)
            + |Δ cos 2θ| + |Δ sin 2θ|  pondéré par (1 − B/A) ]
```

Deux choix méritent une explication. Le grand axe est comparé en **logarithme** : se tromper de 20 px sur un cerceau de 100 px est grave, sur un cerceau de 800 px beaucoup moins, et le logarithme traduit exactement cette erreur relative. L'inclinaison est pondérée par **(1 − B/A)** : pour une ellipse presque ronde, l'angle n'a pas de sens et ne doit pas être pénalisé.

5.3 Réglages

Réglage	Valeur	Rôle
Optimiseur	AdamW, pas 10^{-3} (tête), 10^{-4} (backbone)	Le backbone pré-entraîné apprend 10 fois plus lentement
Calendrier	Cosinus sur 50 époques	Grandes corrections au début, ajustements fins ensuite
Arrêt anticipé	12 époques sans progrès	On garde le meilleur modèle, pas le dernier
Lot	16 images, environ 30 s par époque sur CPU	≈ 25 minutes par entraînement

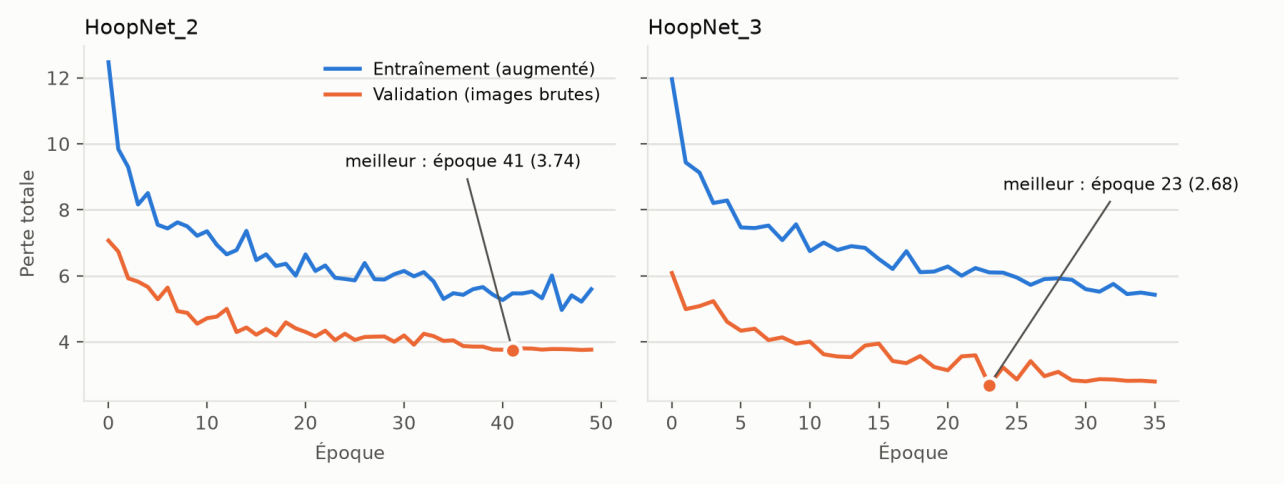


Figure 2 – Perte d'entraînement et de validation. Les valeurs ne se comparent pas d'un graphique à l'autre : les deux modèles n'ont ni les mêmes étiquettes ni la même validation.

Pourquoi la validation est-elle plus basse que l'entraînement ? Ce n'est pas une anomalie : les images d'entraînement sont volontairement rendues difficiles (rotations, collages, flou, bruit) et le Dropout est actif, alors que la validation utilise des images brutes et le réseau complet. HoopNet_2 a progressé jusqu'à l'époque 41 ; HoopNet_3 a atteint son meilleur point dès l'époque 23, puis s'est arrêté à l'époque 35.

6. Résultats

Chaque modèle est évalué sur **sa propre** validation, faite d'images qu'il n'a jamais vues. Les deux colonnes sont donc honnêtes chacune de leur côté, mais elles ne se comparent qu'avec prudence : les étiquettes et la composition de la validation ont changé entre les deux (voir aussi le §7).

Mesure (validation)	HoopNet_2 (époque 41)	HoopNet_3 (époque 23)	Lecture pour le vol
Exactitude de présence	83,7 %	87,4 %	V3 se trompe moins sur « cerceau ou pas »
Précision / rappel	86,0 % / 90,2 %	86,4 % / 93,3 %	Le rappel élevé : il manque rarement un cerceau
Erreur du centre	68 px	86 px	≈ 4 à 5° de cap : encore trop
Erreur sur le grand axe	10,8 %	11,3 %	≈ 11 % d'erreur sur la distance
Erreur sur B/A	0,075	0,074	Voir la limite physique au §8
Erreur d'inclinaison	11,3° (26 ellipses)	2,8° (17 ellipses)	Peu d'ellipses aplaties : chiffre fragile

6.1 Ce que montrent les galeries



Figure 3 – HoopNet_2 sur sa validation. Vert : ellipse annotée ; rouge : prédiction ; p : probabilité de présence. Deux premières lignes : exemples réussis ; dernière ligne : erreurs de présence.



Figure 4 – HoopNet_3 sur sa validation, même présentation.

- **Les cerceaux bien visibles sont trouvés**, et l'ellipse prédite a en général la bonne taille et la bonne forme, y compris pour les cerceaux qui débordent de l'image.

- **Les cerceaux manqués** sont surtout ceux vus presque de profil, collés au support et au filet : l'anneau n'y est plus qu'un trait vertical, très difficile à distinguer du poteau en bois.
- **Un biais systématique du centre.** Mesuré sur les 70 cerceaux détectés de la validation de HoopNet_3, le centre prédit est en moyenne décalé de **+32 px vers la droite et +30 px vers le bas**, et 66 % des prédictions tombent à droite de l'annotation. Ce décalage constant représente une bonne part de l'erreur du centre (écart-type autour de cette moyenne : 80 px en x, 67 px en y).

> **Piste.** un biais constant n'est pas du bruit : il a une cause. Pistes à vérifier : une convention de coordonnées différente entre l'annotation, le miroir et le redimensionnement (pixel = coin ou centre), ou le support en bois, toujours sous le cerceau, qui « tire » la prédiction vers le bas. Un test simple : évaluer le modèle sur les images retournées horizontalement. Si le biais change de signe, c'est le réseau ; s'il reste à droite, c'est une convention de coordonnées.

7. Le piège de la fuite de données

En voulant comparer V1, V2 et V3 sur la même validation, on a découvert que le tableau obtenu était faux : **107 des 127 images de validation de HoopNet_3 avaient servi à entraîner HoopNet_2**. HoopNet_2 les avait donc déjà vues, et ses scores étaient artificiellement bons.

La cause était le découpage entraînement / validation. Il regroupait les images en blocs par classe, puis mélangeait les blocs au hasard. Dès qu'une image changeait de classe (ce qu'ont fait la revue de V2 et la règle de V3), tous les blocs étaient renumérotés et le mélange donnait un découpage entièrement différent.

La correction (déjà dans data.py) : chaque image appartient à une tranche de temps fixe de 5 secondes, et le côté de la tranche est fixé par un hachage de son numéro. Réétiqueter une image ne déplace plus aucune autre image : on l'a vérifié en simulant le réétiquetage de 34 images. La nouvelle validation compte 120 images (90 cerceaux, 30 vides), couloir et cage compris.

> **Leçon.** une comparaison entre deux modèles n'a de sens que sur une validation **identique** et que **ni l'un ni l'autre** n'a vue. C'est pourquoi HoopNet_4 doit être réentraîné : il servira de référence, et toutes les expériences suivantes (nouvelles données, autre architecture, autre perte) seront mesurées sur la même validation stable.

8. De l'ellipse au cerceau 3D

Le module pose.py transforme l'ellipse en position et orientation du cerceau, à partir de la calibration et du diamètre de 68 cm. Les rayons qui partent de la caméra et touchent le cerceau forment un cône dont l'équation se lit sur l'ellipse ; seuls deux plans coupent ce cône selon un cercle de 34 cm de rayon, d'où **deux solutions** (l'ambiguïté double), départagées en suivant la normale d'une image à l'autre ou en utilisant le fait que le cerceau est vertical.

Sur 500 cerceaux simulés avec la vraie calibration, l'erreur maximale est de 0,4 mm sur la position et de 0,13° sur la normale : la géométrie est exacte, toute l'erreur viendra du réseau. Une limite physique compte cependant pour le guidage : près de la face, B/A ≈ cos(angle) varie très peu.

Vrai angle de vue	B/A	Angle estimé avec une erreur de 0,07 sur B/A
0°	1,000	0 à 22°
20°	0,940	0 à 30°
40°	0,766	33 à 46°

Loin de l'axe du cerceau, la normale est fiable et le drone peut tourner franchement ; près de l'axe, il vaut mieux centrer le cerceau dans l'image et avancer.

9. Prochaines étapes

Priorité	Action	Pourquoi
1	Réentraîner HoopNet_4 sur le découpage stable	Disposer d'une référence mesurée honnêtement
2	Chercher la cause du biais du centre (+32 px, +30 px)	Un biais constant est souvent une erreur corrigible ; il pèse lourd dans l'erreur du centre
3	Capturer des cerceaux vus de profil et en bord d'image	Ce sont les cas manqués actuellement
4	guidance.py : pose 3D → consignes pour la boucle du collègue	Relier la vision au contrôle
5	live_demo.py : flux Tello → CNN → pose, moteurs coupés	Tester toute la chaîne avant de voler

Annexe – Commandes

```
cd ~/Documents/tello && source venv/bin/activate
python -m tellovision.main configs/detect.yaml train     # entraîner (crée logs/HoopNet_N)
python -m tellovision.main logs/HoopNet_3 test           # réévaluer un modèle + galerie
python tools/show_augmentations.py configs/detect.yaml   # voir les augmentations
python tools/annotate_hoops.py --files captures/a_verifier.txt   # revoir une liste d'images
tensorboard --logdir logs                                 # courbes dans le navigateur
```